



COMPUTER SCIENCE 180A ALGORITHMS

PROGRAMMING ASSIGNMENT 2 – CUSTOM HUFFMAN

We are using custom file formats (.one, .lla, .ant) to carry bytes around.
You DO NOT need to understand how data is laid out in these files.
However, do note that data is described per byte in each file.

You will implement Huffman algorithm to support compression and decompression.
Following are the implementations that you have to do on Huffman.py

1. Huffman Tree Node Initialization

Node.__init__(self, freq, sym, left, right) {

Arguments:

self	Node	-
freq	integer	Frequency of a symbol
sym	integer?	Symbol
left	Node?	Left child node
right	Node?	Right child node
uid	integer	Unique Identifier of Node
min_sym	integer	minimum of min_sym of children

Initialize Node.

min_sym should be set to sym if leaf node.

Return None -

}

2. Huffman Tree Node Less Than

Node.__lt__(self, other) {

Arguments:

self	Node	-
other	Node	-

Compare and return Boolean if self is LESS THAN other

To avoid any equal, implement multiple comparisons with following order

1. frequency 2. min_sym 3. uid

Return Bool is self less than other

}

3. Build Huffman Tree

`build_tree(frequency) {`

Arguments:

frequency [integer] list of 256 integers representing byte frequency

From frequency list, create a list of leaf nodes for all bytes with non-zero frequency

From list of leaf nodes, create huffman tree. Use `list.sort()` for deterministic outcome

To sort list, use `.sort()` after implementing `Node.__lt__()` method.

Edge cases : empty input, single symbol

Return Node root of the tree

`}`

4. Derive Binary Codes

`derive_codes(root, frequency) {`

Arguments:

root Node root node of Huffman tree

frequency [Integer] list of 256 integers representing byte frequency

Build dictionary that represents binary codes as { symbol : binary code, ... }

Edge cases : empty tree, single symbol

4.5 DFS on Huffman Tree

`dfs(node, code) {`

Arguments:

node Node node

code string current code string (accumulated 0s or 1s)

Implement recursively.

When reached leaf node, populate the dictionary. else, recurse.

Left is 0, Right is 1.

Return None -

`}`

Return Dictionary Dictionary of any non-zero bytes and its binary code

`}`

5. Count Frequencies of Byte

`count_frequencies(path) {`

Arguments:

path str path to the file

```

    Count frequencies of each byte and populate frequencies list
    Read file by chunk size (1MB), and iterate by a byte

    Return          list, int          frequencies, total file size

}

```

To test your code

Test your implementations

```
python3 tests/test.py
```

View .one file

```
tests/one_viewer.py
```

```
python3 tests/one_viewer.py examples/example.one
```

Compress .one to .ant

```
python3 src/one_to_ant.py examples/example.one examples/example.ant
```

Decompress .ant to .one

```
python3 src/ant_to_one.py examples/example.ant examples/examples2.one
```

Compare example.one and example2.one

```
diff examples/example.one examples/example2.one
```

Compress .txt to .lla

```
python3 src/txt_to_lla.py examples/example.txt examples/example.lla
```

Decompress .lla to .txt

```
python3 src/lla_to_txt.py examples/example.lla examples/example2.txt
```

Compare example.txt and example2.txt

```
diff examples/example.txt examples/example2.txt
```

What to submit

Implemented **huffman.py**

Compressed a.txt to **a.lla**

Compresses white.one to **white.ant**

Decompressed brandeis.ant to **brandeis.one**

Decompressed PA2.lla to **PA2.txt**

Put all **.ant.one.lla** .txt into **output/** directory

compress your directory into **firstname_lastname_PA2.zip**

submit **firstname_lastname_PA2.zip**